

MDCGS部署手册

1. 概述

1.1 系统架构组件

1.2 环境要求

2. 部署准备

2.1 安装 Docker 和 Docker Compose

2.2 获取部署文件

2.3 准备外部依赖

3. Docker 部署

3.1 环境变量配置

3.2 部署步骤

3.3 如未准备好 MySQL / Redis

4. 服务管理

4.1 常用 Docker Compose 命令

4.2 数据持久化

5. 常见问题

5.1 数据库连接失败

5.2 Redis 连接失败

5.3 前端页面空白或 API 报 502

5.4 首次部署无用户数据

6. 附录

6.1 端口说明

6.2 项目文件结构

文档版本：v1.0

更新时间：2026-07-31

系统名称：MDCGS（Metadata Classification & Grading System）数据分类分级管理系统

1. 概述

本文档详细描述了 MDCGS 系统的 Docker 部署方案。系统采用前后端分离架构，前端基于 Vue 3 + Element Plus，后端基于 FastAPI，数据存储使用 MySQL + Redis，并通过 Celery 实现异步任务处理。

1.1 系统架构组件

组件	技术栈	说明
前端	Vue 3 + Element Plus + TypeScript + Vite + ECharts	运行于 Nginx 容器
后端	Python 3.11 + FastAPI + SQLAlchemy	FastAPI 应用服务
数据库	MySQL 8.x	业务数据持久化
缓存/队列	Redis 7.x	缓存、Celery 消息队列
异步任务	Celery (pool=threads)	数据扫描、分类分级任务
反向代理	Nginx	静态资源托管、API 反向代理

1.2 环境要求

资源项	最低配置	推荐配置
CPU	2 核	4 核及以上
内存	2 GB	4 GB 及以上
磁盘	20 GB	50 GB 及以上
Docker	20.10+	24.0+
Docker Compose	v2.0+	v2.20+

2. 部署准备

2.1 安装 Docker 和 Docker Compose

```
1 # 安装 Docker (CentOS)
2 sudo yum install -y yum-utils
3 sudo yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
4 sudo yum install -y docker-ce docker-ce-cli containerd.io
5 sudo systemctl start docker
6 sudo systemctl enable docker
7
8 # 安装 Docker (Ubuntu/Debian)
9 sudo apt-get update
10 sudo apt-get install -y docker.io docker-compose-v2
11
12 # 验证安装
13 docker --version
14 docker compose version
```

2.2 获取部署文件

```
1 # 克隆项目代码
2 git clone https://github.com/HaoY-l/mdcgs.git
3 cd mdcgs
4
5 # 查看部署相关文件
6 ls -la docker-compose.yml Dockerfile nginx.conf
```

2.3 准备外部依赖

系统需要以下外部服务（建议使用云服务或独立部署）：

服务	用途	推荐方案
MySQL 8.x	业务数据存储	云数据库 RDS
Redis 7.x	缓存 & Celery 消息队列	云 Redis

3. Docker 部署

3.1 环境变量配置

创建 `.env` 文件，放置在 `docker-compose.yml` 同级目录：

```
1  # ===== MySQL (必填) =====
2  DB_HOST=Mysql主机地址,不要写127.0.0.1
3  DB_PORT=Mysql端口
4  DB_USER=Mysql用户名
5  DB_PASSWORD=Mysql密码
6  DB_NAME=mdcgs
7  DATABASE_URL=mysql+pymysql://用户名:密码.com@host,不要写127.0.0.1:端口/mdcgs
8
9
10 # ===== Redis (必填,不要写127.0.0.1) =====
11 REDIS_URL=redis://Redis主机地址:6379/0
12 REDIS_PASSWORD=Redis密码
13 CELERY_BROKER_URL=redis://Redis密码@Redis主机地址:6379/1
14 CELERY_RESULT_BACKEND=redis://Redis密码@Redis主机地址:6379/2
15
16 # ===== 安全 (必填, 建议随机生成) =====
17 SECRET_KEY=your-random-secret-key-here-change-me
18 ENCRYPT_KEY=your-32-byte-encryption-key-here
```

安全提醒：生产环境请务必修改 `SECRET_KEY` 和 `ENCRYPT_KEY` 为随机生成的强密钥，避免使用默认值。部署统一宿主机时，主机地址禁止禁用“127.0.0.1”。

3.2 部署步骤

```

1  # 1. 进入项目目录
2  cd /path/to/mdcgs
3
4  # 2. 创建 .env 文件 (按 3.1 节配置)
5  vim .env
6
7  # 3. 启动所有服务
8  docker compose up -d
9
10 # 4. 查看服务启动状态
11 docker compose ps
12
13 # 5. 查看日志
14 docker compose logs -f
15
16 # 6. 访问系统
17 # 前端地址: http://<服务器IP>:7785
18 # 后端 API: http://<服务器IP>:8000/api/v1
19 # 健康检查: http://<服务器IP>:8000/api/v1/health
20
21 # 7. 首次登录
22 # 用户名: admin
23 # 密码: admin123

```

3.3 如未准备好 MySQL / Redis

快速创建 MySQL + Redis 容器

```

1  # MySQL
2  docker run -d --name mysql -p 3306:3306 \ -e MYSQL_ROOT_PASSWORD=123456 \
-e MYSQL_ROOT_HOST=% mysql
3
4  # 创建数据库
5  docker exec -it mysql mysql -uroot -p123456 \ -e "CREATE DATABASE IF NOT E
XISTS mdcgs DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
6
7  # Redis
8  docker run -d --name redis -p 6379:6379 \ --restart always redis:7-alpine
\ redis-server --requirepass "123456" --appendonly yes

```

Docker 镜像加速 (如果拉取镜像失败可尝试)

```
1  sudo mkdir -p /etc/docker && sudo tee /etc/docker/daemon.json <<EOF
2  {
3  "registry-mirrors": [
4      "https://docker.xuanyuan.me",
5      "https://docker.lms.run",
6      "https://docker.m.daocloud.io",
7      "https://docker.lpanel.live",
8      "https://docker.hlmirror.com",
9      "https://hub.rat.dev",
10     "https://docker.mirrors.ustc.edu.cn",
11     "https://docker-0.unsee.tech"
12 ]
13 }
14 EOF
15 sudo systemctl daemon-reload && sudo systemctl restart docker
```

4. 服务管理

4.1 常用 Docker Compose 命令

```
1  # 启动所有服务
2  docker compose up -d
3
4  # 停止所有服务
5  docker compose down
6
7  # 重启特定服务
8  docker compose restart mdcgs-frontend
9  docker compose restart mdcgs-backend
10
11 # 查看日志（实时）
12 docker compose logs -f mdcgs-backend
13 docker compose logs -f mdcgs-frontend
14
15 # 重新构建并启动（代码变更后）
16 docker compose up -d --build mdcgs-frontend
```

4.2 数据持久化

后端服务的持久化数据挂载在 `backend_data` 命名卷中，包含以下目录：

目录	用途	路径（容器内）
data/uploads	上传文件	/app/data/uploads
data/exports	导出报告	/app/data/exports
data/backups	系统备份	/app/data/backups
data/logs	运行日志	/app/data/logs

5. 常见问题

5.1 数据库连接失败

现象：后端启动后连接 MySQL 失败。

排查步骤：

1. 检查 `.env` 中数据库地址、端口、账号密码是否正确
2. 确认 MySQL 服务允许远程连接
3. 检查防火墙是否开放对应端口
4. `docker compose logs backend` 查看详细错误日志

5.2 Redis 连接失败

现象：任务无法运行，Celery Worker 报错。

解决：

1. 检查 Redis 密码配置是否与 `.env` 一致
2. 确认 Redis 服务可达（ping 测试）
3. 检查 `REDIS_PASSWORD` 配置后，系统会主动注入密码到 URL 中

5.3 前端页面空白或 API 报 502

现象：访问前端页面显示空白，或浏览器控制台显示 API 返回 502。

排查：

1. 确认后端容器是否已启动：`docker compose ps`

2. 确认 Nginx 配置中 `proxy_pass http://backend:8000` 的 `backend` 与 `docker-compose` 中 `service` 名称一致
3. 确认网络是否正常: `docker compose exec frontend ping backend`

5.4 首次部署无用户数据

现象：登录页面提示系统未初始化。

说明：后端首次启动时，若检测到数据库无用户记录，会自动执行种子数据初始化，创建默认管理员账户。

6. 附录

6.1 端口说明

端口	服务	说明
7785	前端	Nginx 提供前端静态资源访问
8000	后端	FastAPI 后端服务

6.2 项目文件结构

```
1  mdcgs/                                # 前端项目
2  |— docker-compose.yml                 # Docker 编排文件
3  |— Dockerfile                         # 前端构建文件
4  |— nginx.conf                         # Nginx 反向代理配置
5  |— .env                               # 环境变量配置
6  |— src/                               # 前端源码
7  |— dist/                              # 构建产物
```